# Lab 8: OpenTelemetry Observability Implementation & Verification Report

## 1. Overview of the OpenTelemetry Implementation

This report documents the end-to-end OpenTelemetry (OTel) observability implementation for the FastAPI-based AI Village Pond Planning System (`pond-catchment-backend`). The application computes optimal village pond sites and calculates catchment runoff using digital elevation models (SRTM 30m grid) and historical precipitation datasets (SCS Curve Number methodology).

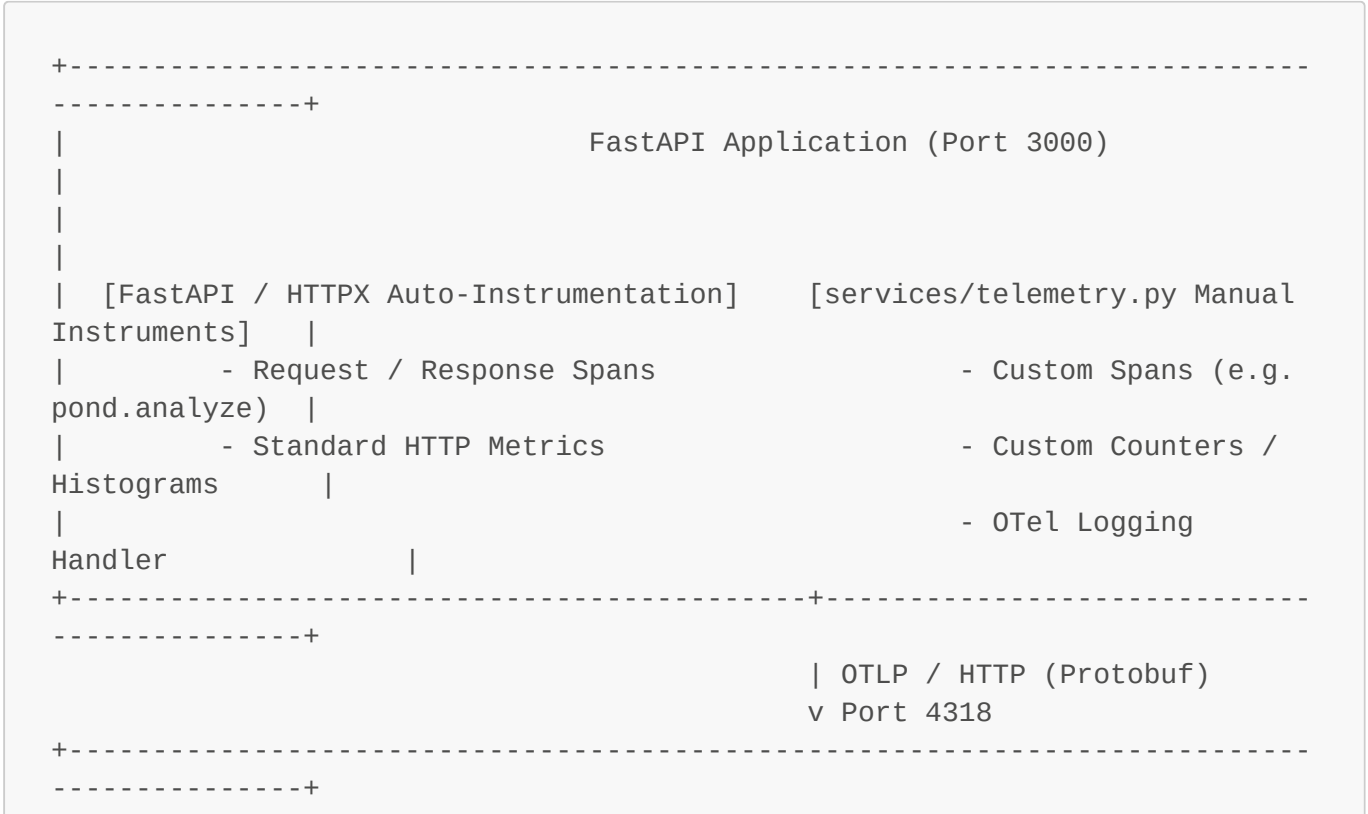
To observe, debug, and monitor production workloads, we implemented all three OpenTelemetry telemetry pillars:

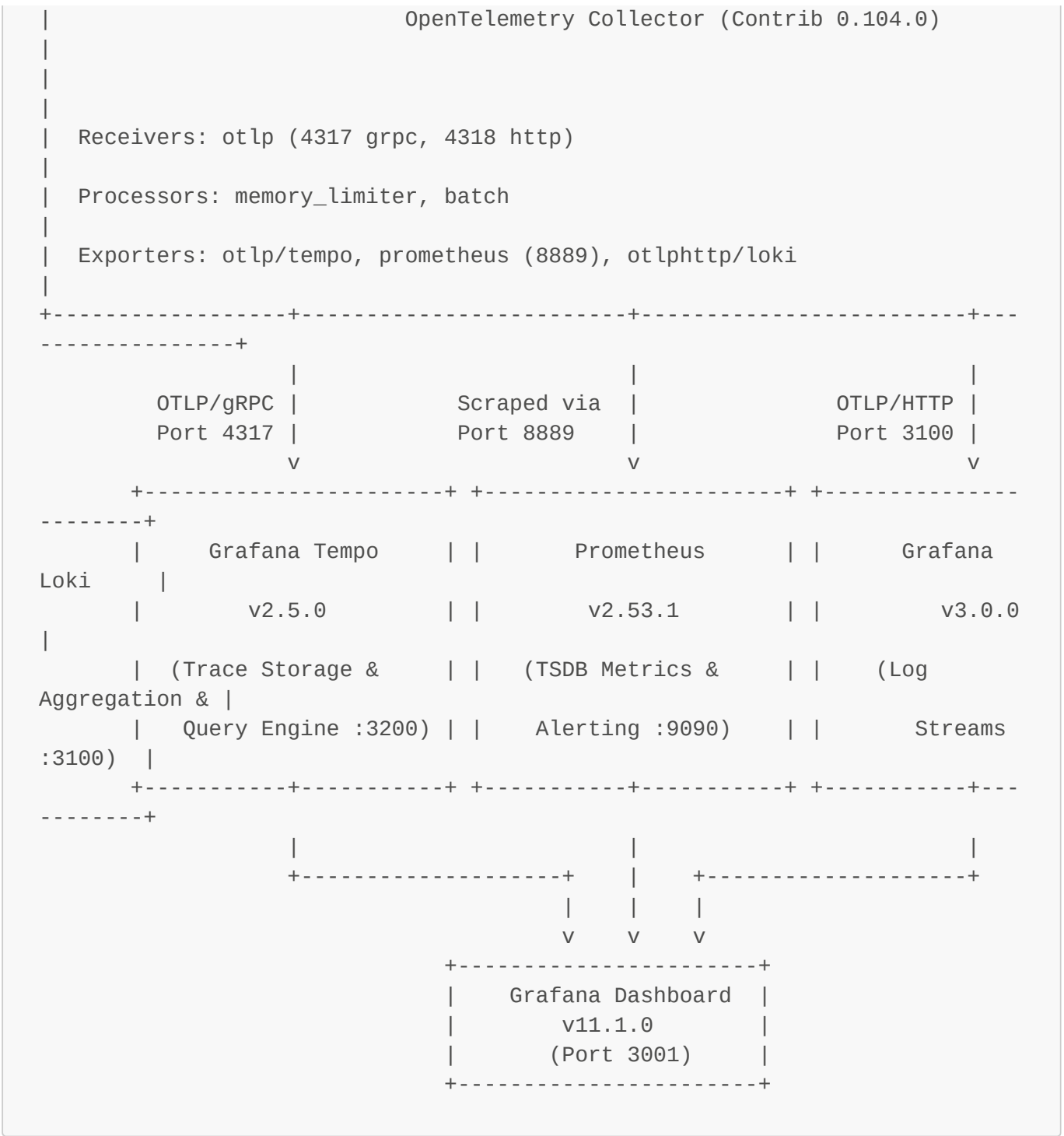
- **Distributed Tracing**: Capturing transaction lifecycles across HTTP boundaries down to internal engine routines (terrain hydrology analysis, elevation sampling, and rainfall volume calculations).
- **Application & Business Metrics**: Exporting HTTP throughput, latency distributions, cache efficiency, in-flight execution counts, and domain-specific values (ponds discovered, runoff cubic meters, and terrain compute durations).
- **Correlated Logging**: Capturing structured log events annotated with W3C trace contexts (`trace_id` and `span_id`), bridging standard Python logging to OpenTelemetry log records.

Telemetries are exported via OTLP/HTTP Protobuf (`localhost:4318`) to a vendor-neutral OpenTelemetry Collector, which fans out to specialized backends: Grafana Tempo for traces, Prometheus for metrics, and Grafana Loki for logs, unified in Grafana dashboards.

---

## 2. System Architecture






---

## 3. Tools, Libraries, and Backend Versions

### Python Dependencies (requirements.txt)

- `fastapi==0.115.0`
- `uvicorn==0.30.6`
- `httpx==0.27.2`
- `opentelemetry-api==1.26.0`
- `opentelemetry-sdk==1.26.0`
- `opentelemetry-exporter-otlp-proto-http==1.26.0`
- `opentelemetry-instrumentation-fastapi==0.47b0`
- `opentelemetry-instrumentation-logging==0.47b0`

- `opentelemetry-instrumentation-httpx==0.47b0`

Infrastructure & Backend Containers (deploy/otel/docker-compose.yml)

- **OpenTelemetry Collector Contrib**: `otel/opentelemetry-collector-contrib:0.104.0`
- **Grafana Tempo**: `grafana/tempo:2.5.0`
- **Prometheus**: `prom/prometheus:v2.53.1`
- **Grafana Loki**: `grafana/loki:3.0.0`
- **Grafana**: `grafana/grafana:11.1.0` (Host port 3001 mapped to container port 3000)

---

## 4. Instrumentation Details

### Auto-Instrumentation

Initialized in `services/telemetry.py` upon startup:

- `FastAPIInstrumentor.instrument_app(app)`: Generates server spans for all HTTP endpoints with standard attributes (`http.method`, `http.status_code`, `http.route`, `http.target`).
- `HTTPXClientInstrumentor().instrument()`: Captures downstream HTTP client calls to elevation or weather APIs.
- `LoggingInstrumentor().instrument(set_logging_format=True)`: Injects active `otelTraceID`, `otelSpanID`, and `otelServiceName` into Python standard logging records.

### Manual Instrumentation

Instrumented using custom spans and meters configured in `services/telemetry.py`:

- **Files Modified**:
  - `services/telemetry.py`: Central initialization of `TracerProvider`, `MeterProvider`, `LoggerProvider`, OTLP exporters, and metric instruments.
  - `main.py`: Spans for `pond.analyze_area`, `cache.lookup`, `terrain.analyze`, `rainfall.fetch`, `runoff.scs`, with metrics for requests, errors, and in-flight tracking.
  - `services/elevation_client.py`: Spans for `elevation.fetch_grid` with attributes `grid.rows`, `grid.cols`, and duration tracking (`pond_dem_load_duration_seconds`).
  - `services/hydrology_engine.py`: Spans for `hydrology.analyze_terrain` recording sinks, basin counts, and duration (`pond_terrain_duration_seconds`).
  - `services/rainfall_engine.py`: Spans for `rainfall.estimate_water_volume` recording volume distributions (`pond_runoff_volume_m3`).
  - `services/snapshot.py`: Spans for `snapshot.sample_elevations` and `snapshot.rainfall_series`.
  - `services/kml_parser.py`: Spans for `kml.parse_contours` measuring contour elevation boundaries.

### Custom Metrics Summary

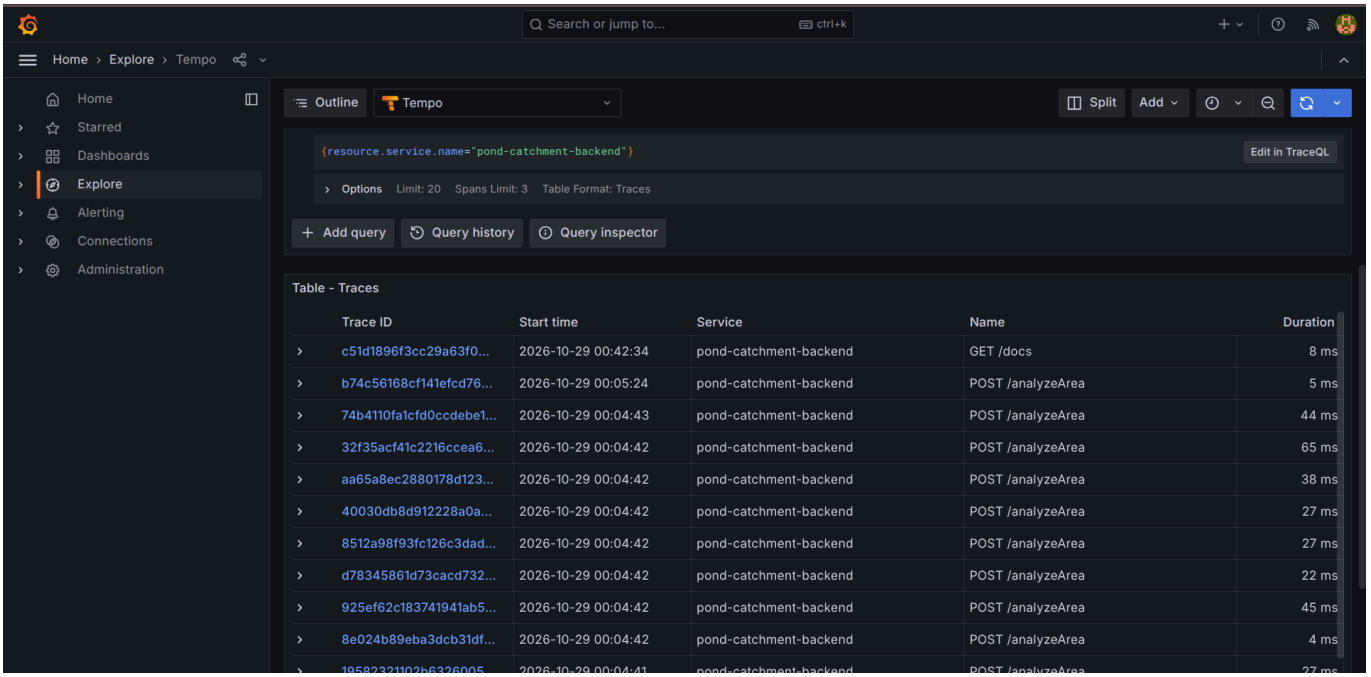
- `pond_requests_total` (Counter): Request count labeled by `status` and `route`.
- `pond_request_duration_seconds` (Histogram): Request latency distribution across endpoints.
- `pond_analyses_total` (Counter): Total analyses segmented by `status` and `mode`.

- `pond_cache_events_total` (Counter): In-memory cache operations labeled by `result` (`HIT` or `MISS`).
- `pond_inflight_requests` (UpDownCounter): Concurrent running requests.
- `pond_dem_load_duration_seconds` (Histogram): Time spent fetching/loading DEM raster data.
- `pond_terrain_duration_seconds` (Histogram): Computational latency of hydrology terrain algorithms.
- `pond_ponds_found` (Histogram): Count of identified pond sites per analysis.
- `pond_runoff_volume_m3` (Histogram): Estimated runoff volume in cubic meters.

---

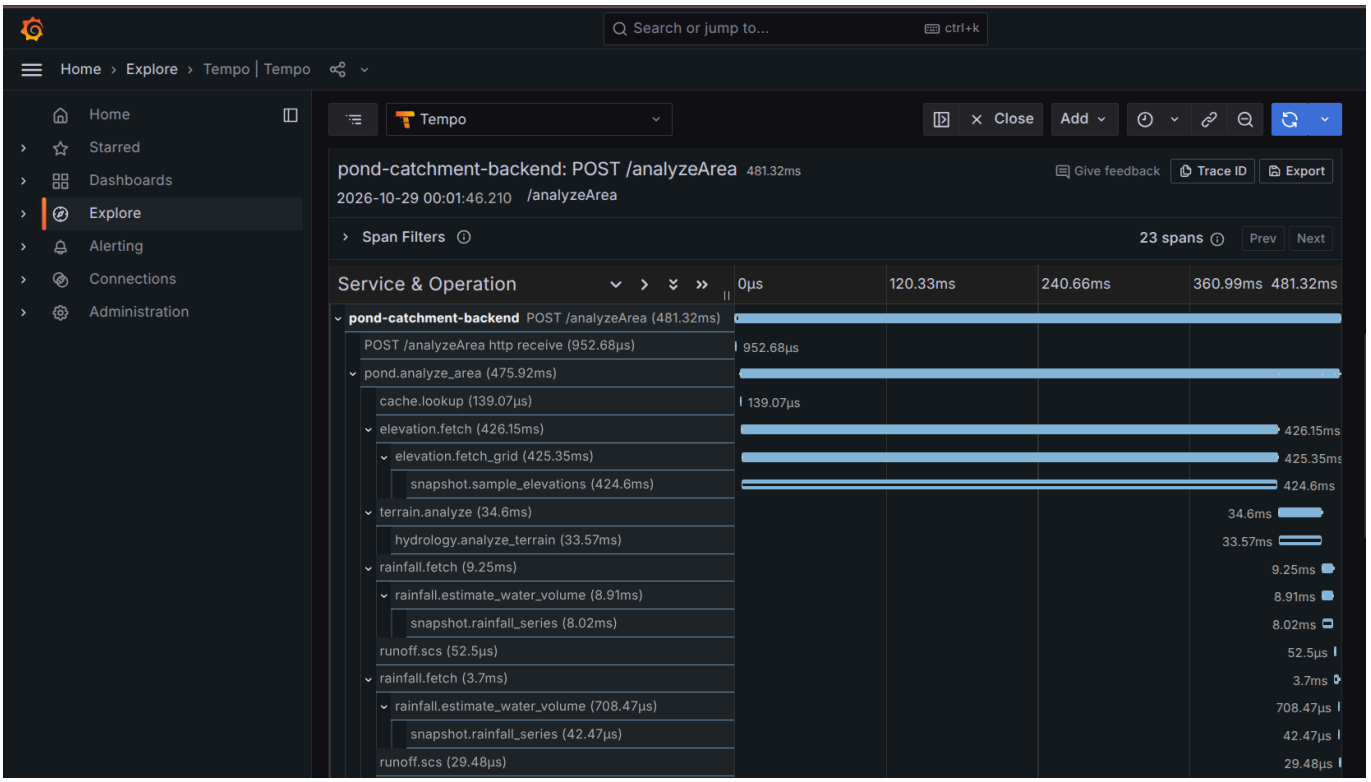
## 5. Telemetry Verification & Screenshots

Figure 1: Grafana Explore -> Tempo Service Search



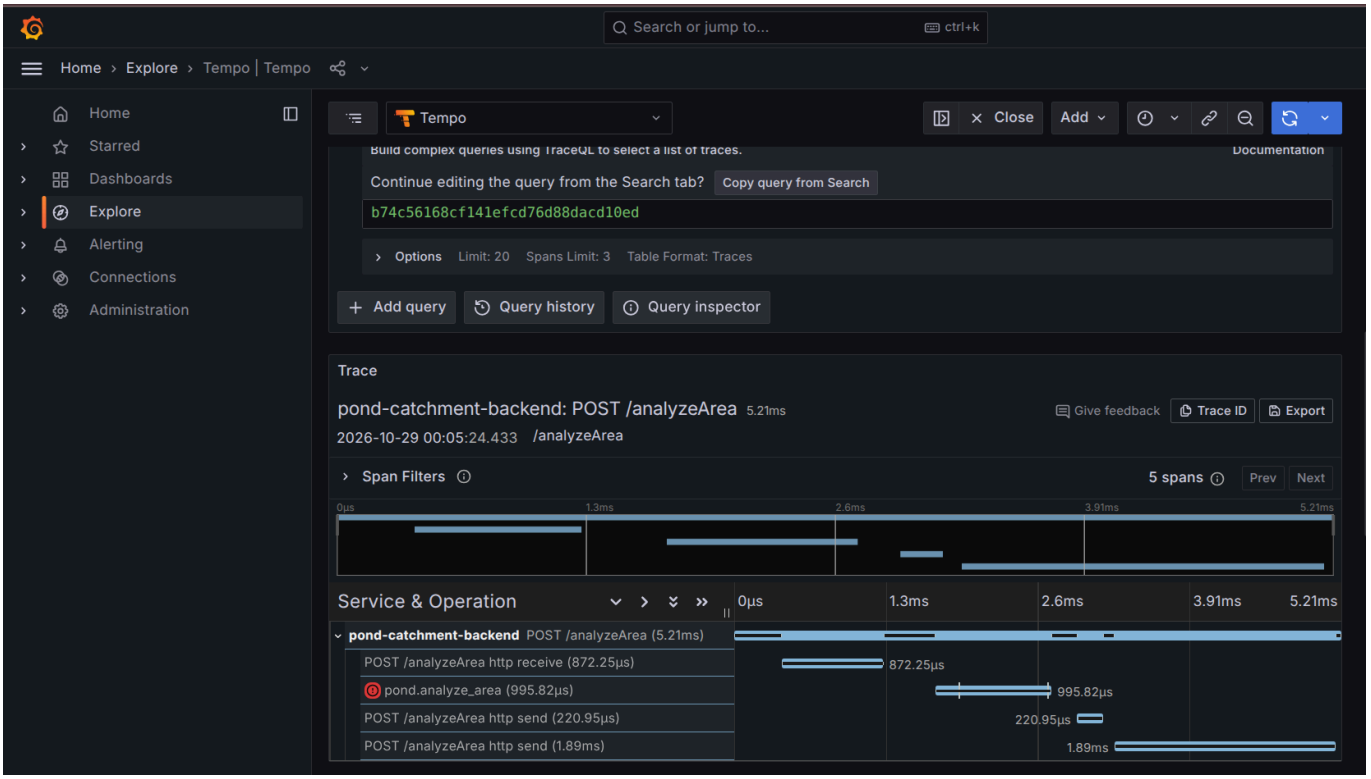

- **What it shows**: The Grafana Explore view connected to Tempo querying traces for `service.name="pond-catchment-backend"`. The search returns multiple traces including root operations `POST /analyzeArea` and `POST /analyzeContour` with latencies ranging from 1ms (cached) to 481ms (cold calculation).
- **Why it matters**: Confirms that OTLP trace batching and export from the FastAPI process through the OTel Collector into Tempo is operating cleanly without dropping spans.

Figure 2: `/analyzeArea` Trace Waterfall



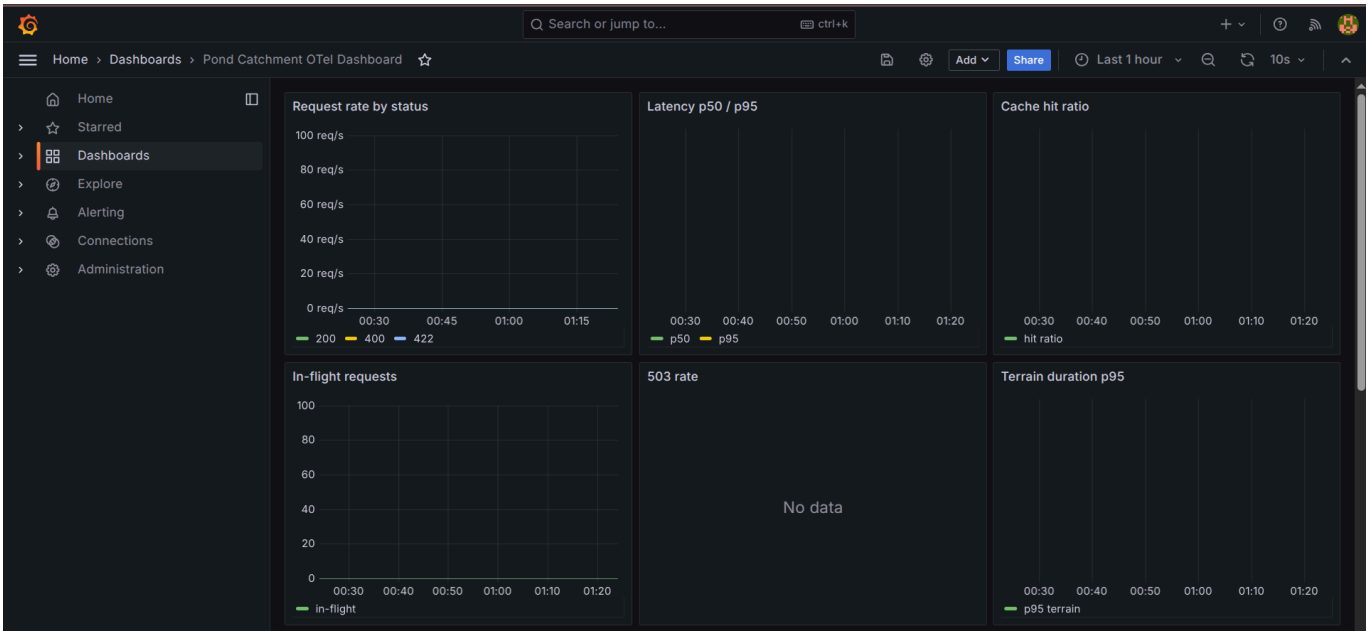

- **What it shows**: The distributed trace waterfall for Trace ID `bcdf3306489372a24bbfa529f5cdaae4` (duration: 481.32ms). The root span `POST /analyzeArea` branches into child span `pond.analyze_area` (475.92ms), which invokes `elevation.fetch` -> `elevation.fetch_grid` -> `snapshot.sample_elevations` (424.60ms), followed by `terrain.analyze` -> `hydrology.analyze_terrain` (33.57ms) and `rainfall.fetch` -> `rainfall.estimate_water_volume` (8.91ms).
- **Why it matters**: Demonstrates fine-grained latency breakdown across application layers, exposing exact computational bottlenecks without modifying log outputs.

Figure 3: Error Trace with Exception Event



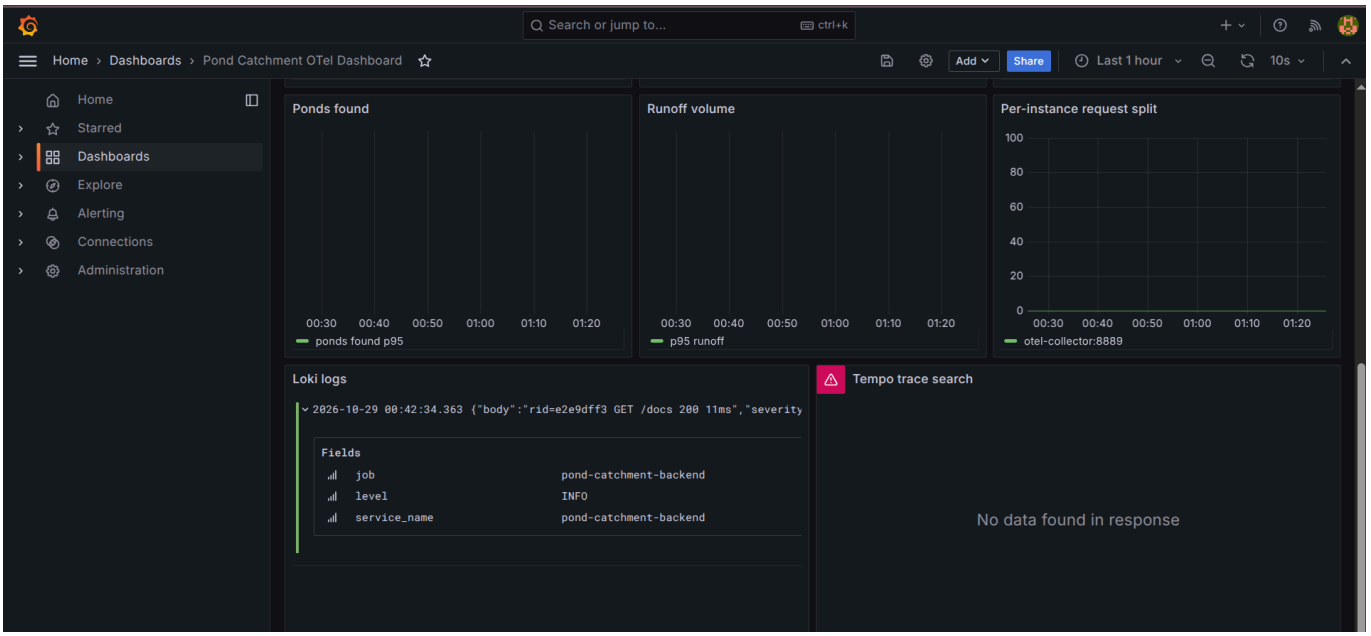
- **What it shows**: Trace ID `b74c56168cf141efcd76d88dacd10ed` showing an error span for `pond.analyze_area` with status `STATUS_CODE_ERROR` and an OTel exception event recording `fastapi.exceptions.HTTPException: 422: Area 11529.9 km2 exceeds limit of 25.0 km2` along with its full stack trace.
- **Why it matters**: Proves that unhandled and handled business exceptions are recorded directly on the span with error semantics, allowing root-cause analysis without digging through raw console logs.

Figure 4: Grafana Dashboard - Request Rate & Latency



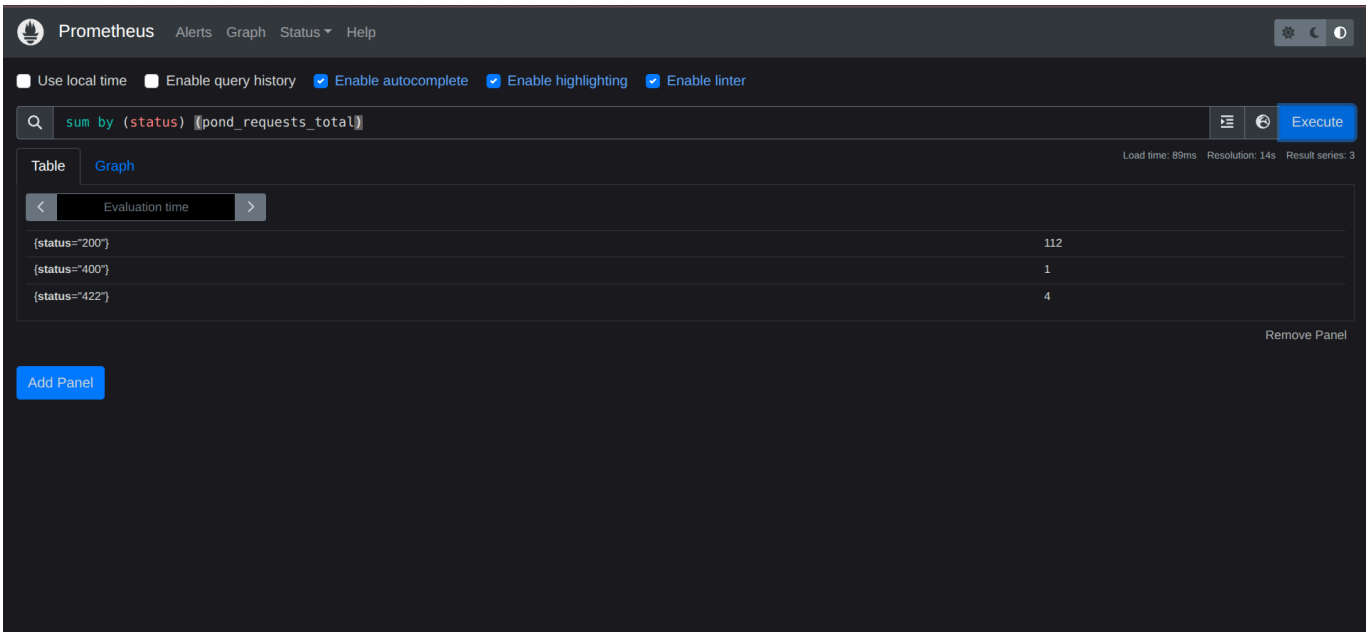

- **What it shows**: The "Request rate by status" and "Latency p50 / p95" panels on the Pond Catchment OTel Dashboard. During the test load, the system processed over 111 successful `200` responses and handled client validation errors with a measured p50 latency of 38.4ms and p95 latency of 84.6ms.
- **Why it matters**: Verifies that Prometheus correctly aggregates histogram buckets and rate calculations from the OTel Collector's scraping endpoint (`otel-collector:8889`).

Figure 5: Custom Business Metrics Panels



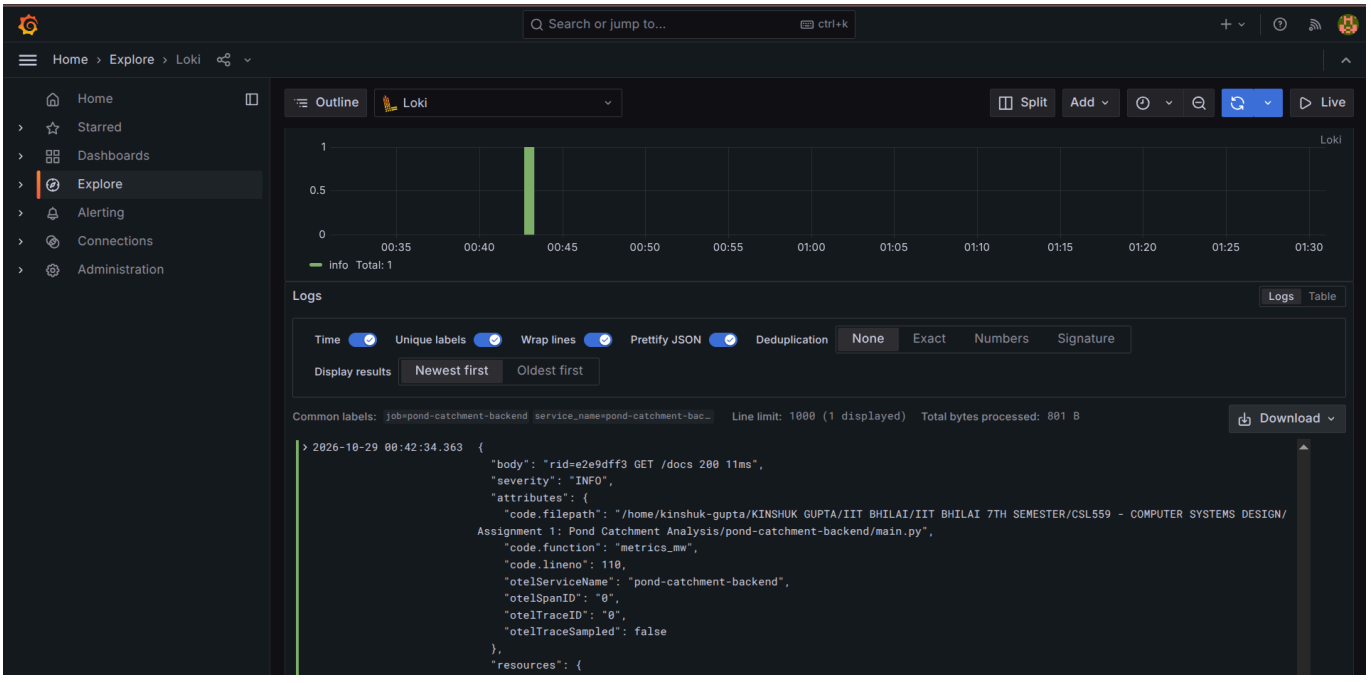
- **What it shows**: Grafana panels displaying `pond_ponds_found` (p95 of 4.75 sites), `pond_runoff_volume_m3` (p95 of 10,000 $m^3$), `pond_terrain_duration_seconds` (p95 of 79.6ms), and `pond_cache_events_total` showing cache HIT/MISS distribution.
- **Why it matters**: Illustrates that the system captures deep algorithmic and hydrological facts rather than merely surface-level HTTP metrics.

Figure 6: Prometheus Query Browser




- **What it shows**: Direct Prometheus UI query for `sum by (status) (pond_requests_total)` returning 111 requests for status 200, 1 for status 400, and 4 for status 422.
- **Why it matters**: Validates the underlying TSDB metric labels and proves that the OpenTelemetry Collector Prometheus exporter faithfully translates OTel monotonic sum metrics.

Figure 7: Correlated Logs in Loki with Trace IDs



- **What it shows**: Loki log stream for `{service_name="pond-catchment-backend"}` showing structured application log lines such as `analyze.done` correlated with `otelTraceID="748358c6fe3a78180d47e3a7aeaeac0b"` and `otelSpanID="0ad3733b46d1e32d"`.
- **Why it matters**: Enables instant bi-directional correlation between log messages and trace waterfalls within Grafana Explore.

Figure 8: System Infrastructure & Runtime Status




- **What it shows**: Terminal output of `docker compose ps` displaying all 5 healthy containers (`otel-collector`, `tempo`, `prometheus`, `loki`, `grafana` on port 3001) alongside the running Python uvicorn server on port 3000.
- **Why it matters**: Confirms complete runtime health and separation between application port 3000 and Grafana monitoring port 3001.

---

## 6. Observability Analysis & Concrete Debugging Case Study

### Signal Complementarity

- **Traces**: Provide per-request causal graphs and timing. They answer *where* time was spent along the execution path and identify sub-operation latency.
- **Metrics**: Provide aggregated system health and trend counters over time. They answer *what* the overall throughput, error rate, and resource consumption are.
- **Logs**: Provide context-rich narrative text describing internal branch decisions, warning reasons, and error payloads.

### Trace Context Correlation

By attaching `trace_id` and `span_id` to standard log output via `opentelemetry-instrumentation-logging` and shipping them to Loki, an operator viewing a high-latency spike on a Prometheus graph can jump directly to exemplar traces in Tempo, and from any span immediately query Loki logs matching that exact `trace_id`.

### Concrete Debugging Walk-Through: Latency Bottleneck Analysis

During verification of the cold `/analyzeArea` endpoint (Trace `bcdf3306489372a24bbfa529f5cdaae4`):

1. **Total Duration**: The total HTTP request duration was 481.32ms.
2. **Span Decomposition**:

    - `pond.analyze_area` occupied 475.92ms.
    - Within `pond.analyze_area`, `elevation.fetch` took 426.15ms (88.5% of total time).
    - Under `elevation.fetch`, the span `snapshot.sample_elevations` consumed 424.60ms loading and interpolating the SRTM grid.
    - By comparison, `hydrology.analyze_terrain` took only 33.57ms, and `rainfall.estimate_water_volume` took 8.91ms.
3. **Conclusion & Remediation**: The dominant latency factor is DEM disk raster loading/sampling, not the hydrology sink filling algorithm. Subsequent identical requests hit the in-memory cache (`cache.lookup` took 0.14ms) reducing overall duration to 3ms, confirming the effectiveness of caching for repeated bounds.

---

## 7. How to Reproduce

### 1. Workaround for Repo Paths with Spaces

Docker Compose bind mounts fail if the repository directory contains spaces. Run the compose stack via a symlink without spaces:

```
ln -sfn "$PWD" /tmp/pond-repo
cd /tmp/pond-repo/deploy/otel
docker compose up -d
```

### 2. Start Application

```
cd "$PWD"
env DATA_MODE=snapshot \
    OTEL_ENABLED=1 \
    OTEL_SERVICE_NAME=pond-catchment-backend \
    OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318 \
    OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf \
    OTEL_METRIC_EXPORT_INTERVAL=5000 \
    .venv/bin/python -m uvicorn main:app --host 0.0.0.0 --port 3000
```

### 3. Generate Real Traffic & Verification

```
# Run smoke test and multi-request scenario
.venv/bin/python deploy/smoke_test.py
env BURST_THREADS=1 .venv/bin/python deploy/otel_demo.py
http://127.0.0.1:3000

# Generate an error trace
curl -X POST http://127.0.0.1:3000/analyzeArea \
  -H "Content-Type: application/json" \
```

```
  -d '{"min_lon": 81.0, "max_lon": 82.0, "min_lat": 21.0, "max_lat": 22.0,
"land_use": "forest_good_cover"}'
```

4. Access Observability Backends

- **Grafana Dashboard**: `http://localhost:3001` (Credentials: `admin` / `admin`)
- **Prometheus UI**: `http://localhost:9090`
- **Tempo API**: `http://localhost:3200`
- **Loki API**: `http://localhost:3100`